

EXPERIMENT 3:

Aim

To write a Python program that solves the Water Jug Problem using the Breadth-First Search (BFS) algorithm.

Objective

To represent the states of two water jugs with given maximum capacities.

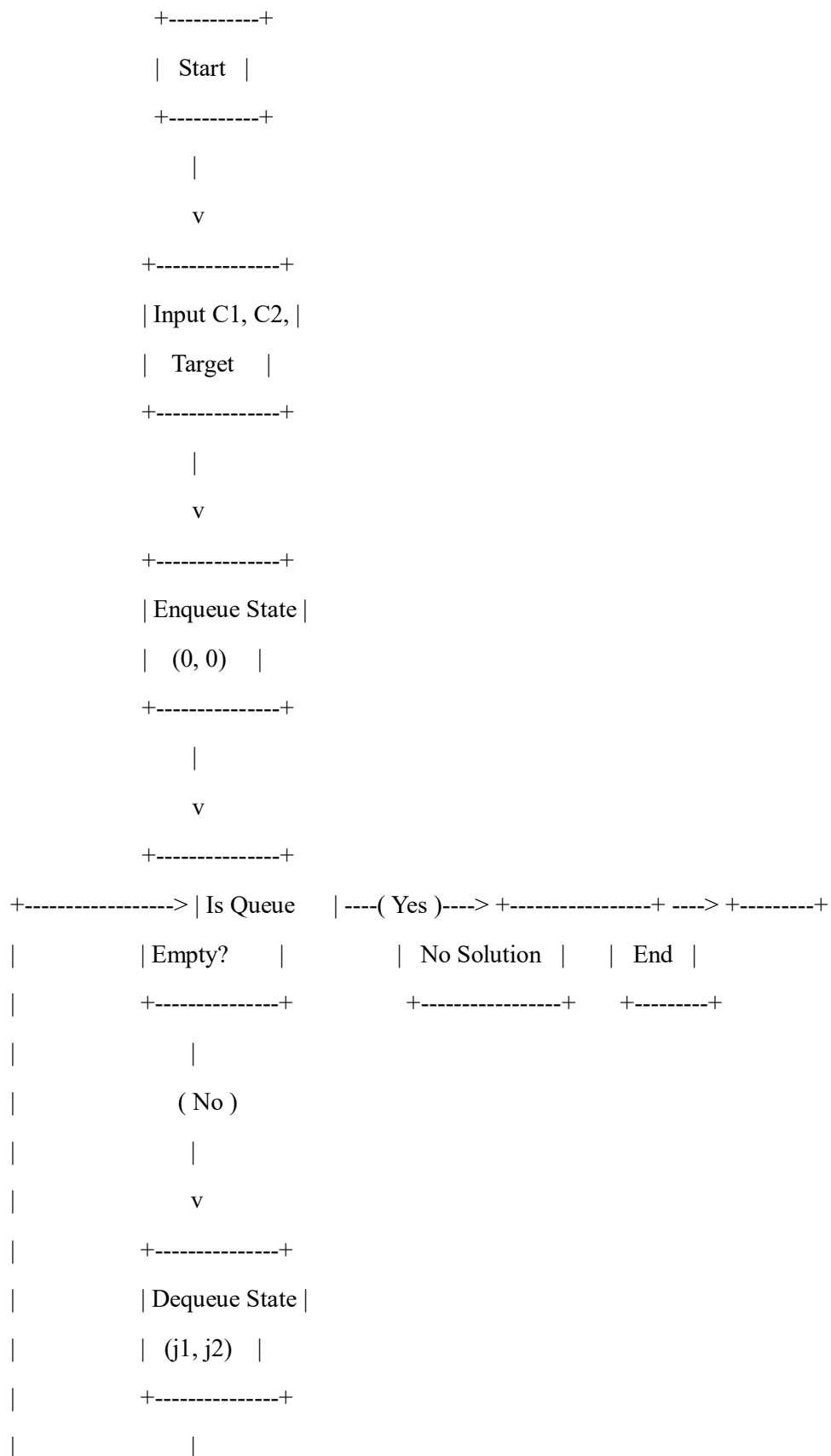
To simulate state transitions by filling, emptying, and pouring water between the two jugs.

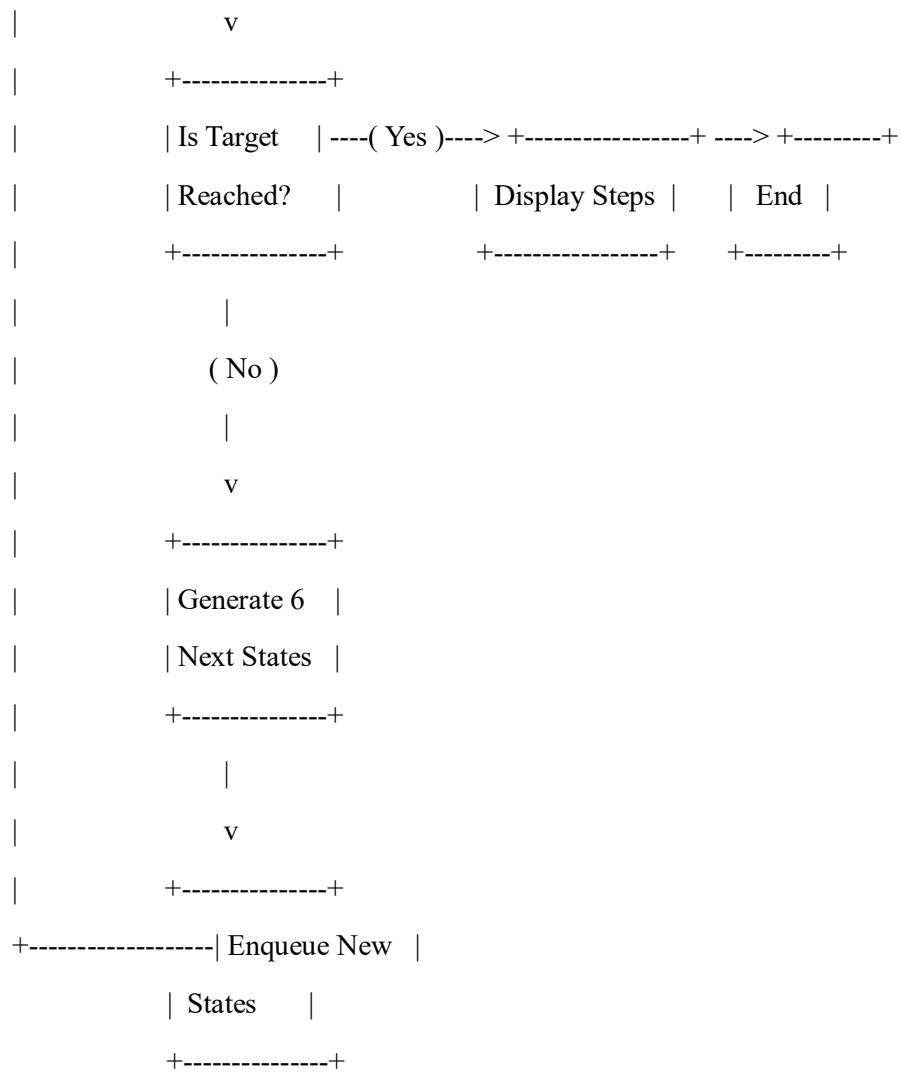
To find and display the step-by-step sequence of moves required to measure a target amount of water.

Algorithm

1. Define the capacities of Jug 1 (C_1), Jug 2 (C_2), and the target amount (T).
2. Initialize an open_list (queue) storing tuples of current states (jug1, jug2) and path histories, and a visited set.
3. Enqueue the initial state (0, 0) with an empty path.
4. Loop while the queue is not empty:
5. Dequeue the front state (j1, j2) and its associated path.
6. If either $j1 == target$ or $j2 == target$, print the step-by-step path and terminate.
7. If state (j1, j2) has already been visited, skip it; otherwise, add it to visited.
8. Generate all possible valid next states:
9. Fill Jug 1: ($C_1, j2$)
10. Fill Jug 2: (j1, C_2)
11. Empty Jug 1: (0, j2)
12. Empty Jug 2: (j1, 0)
13. Pour Jug 1 to Jug 2: (j1 - pour, j2 + pour)
14. Pour Jug 2 to Jug 1: (j1 + pour, j2 - pour)
15. Enqueue all unvisited next states into the queue.

Flowchart





Code

```
from collections import deque
```

```
def solve_water_jug(c1, c2, target):
```

```
    queue = deque([((0, 0), [])])
```

```
    visited = set()
```

```
    while queue:
```

```
        (j1, j2), path = queue.popleft()
```

```
        if (j1, j2) in visited:
```

```
            continue
```

```
        visited.add((j1, j2))
```

```
        current_path = path + [(j1, j2)]
```

```
        if j1 == target or j2 == target:
```

```
            return current_path
```

```
    moves = [
```

```
        (c1, j2),
```

```
        (j1, c2),
```

```
        (0, j2),
```

```
        (j1, 0),
```

```
        (j1 - min(j1, c2 - j2), j2 + min(j1, c2 - j2)),
```

```
        (j1 + min(j2, c1 - j1), j2 - min(j2, c1 - j1))
```

```
    ]
```

```
    for state in moves:
```

```
        if state not in visited:
```

```
            queue.append((state, current_path))
```

```

return None

if name == "main":
    c1, c2, target = 4, 3, 2

    print(f"Solving Water Jug Problem for Jug 1 ({c1}L), Jug 2 ({c2}L), Target ({target}L)...\n")

    solution = solve_water_jug(c1, c2, target)

    if solution:
        print("Steps to achieve target:")
        for step, (j1, j2) in enumerate(solution):
            print(f"Step {step}: Jug 1 = {j1}L, Jug 2 = {j2}L")
    else:
        print("No solution exists for the given capacities and target.")

```

Output

Solving Water Jug Problem for Jug 1 (4L), Jug 2 (3L), Target (2L)...

Steps to achieve target:

Step 0: Jug 1 = 0L, Jug 2 = 0L

Step 1: Jug 1 = 0L, Jug 2 = 3L

Step 2: Jug 1 = 3L, Jug 2 = 0L

Step 3: Jug 1 = 3L, Jug 2 = 3L

Step 4: Jug 1 = 4L, Jug 2 = 2L

Result

The Water Jug program was executed successfully. Using the Breadth-First Search (BFS) algorithm, it determined the optimal sequence of actions to measure the target amount of water.

Conclusion

Breadth-First Search (BFS) provides a systematically complete approach for the Water Jug Problem. By exploring all state transitions layer by layer, it guarantees finding the solution with the minimum number of steps.